PyTorch Overview and Application

CONTENTS

Catalog

01

Introduction to PyTorch

Research tool necessity

01

Agility in saascv

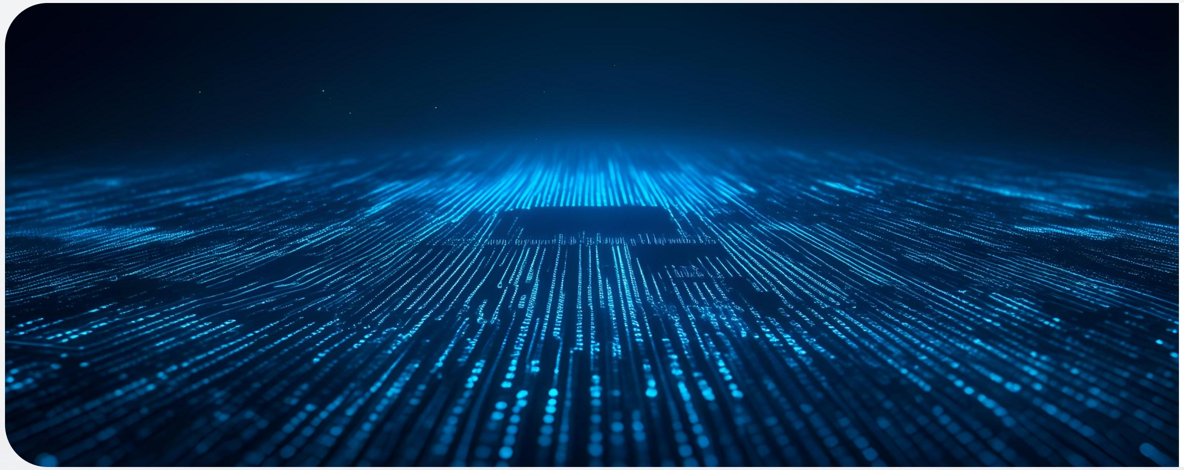
PyTorch's flexibility allows for rapid iteration and experimentation in research.

02

Dynamic computation graphs

PyTorch's dynamic nature enables researchers to build complex models with ease.

Core definition of PyTorch



Python-First framework

PyTorch is designed to be intuitive for Python developers, enhancing productivity.

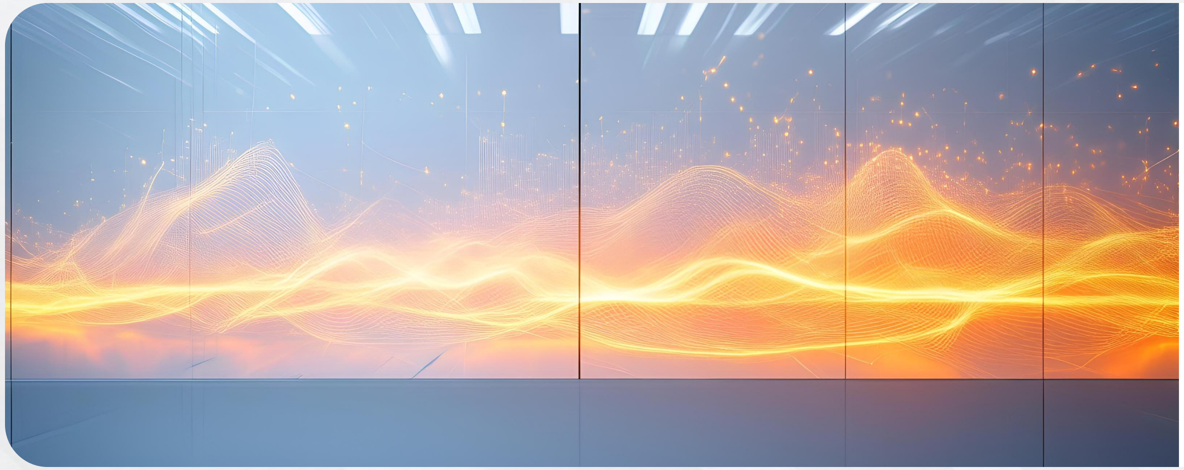
Dynamic framework

PyTorch's dynamic computation graphs support immediate execution, facilitating debugging.

Key difference: dynamic vs. static graph

Dynamic graph flexibility

PyTorch's dynamic graphs allow for more control and ease of use in model development.



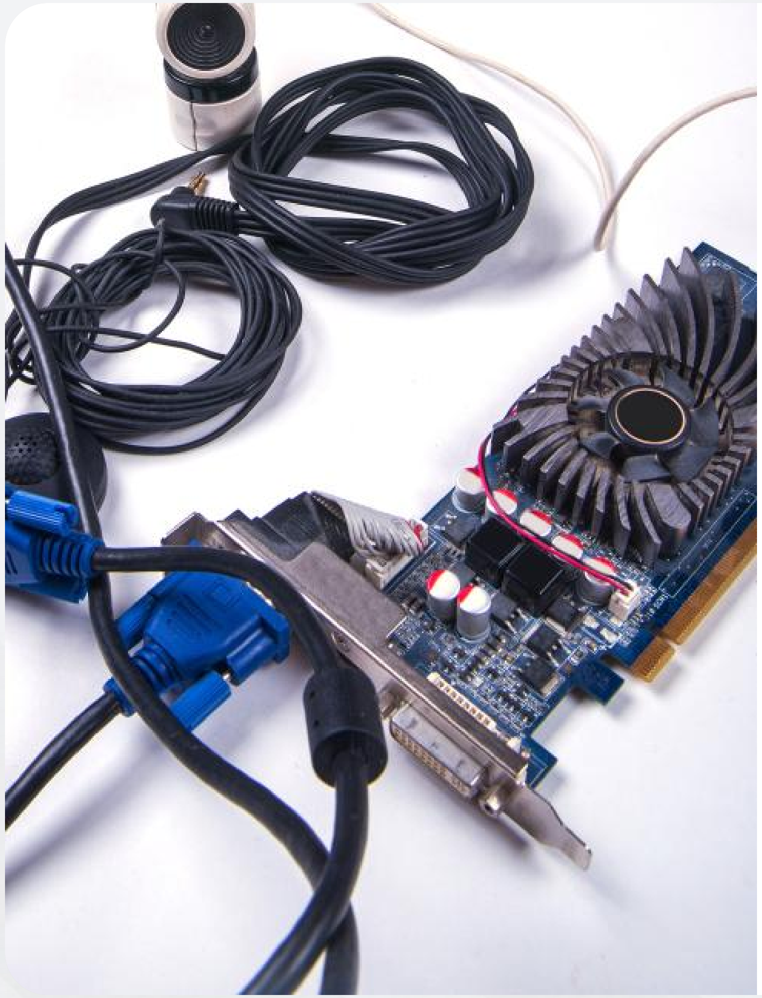
Static graph efficiency

Static frameworks offer computational efficiency and are optimized for production use.

02

Core Data and Engine

Tensors: fundamental data structure



Tensors as NumPy on the GPU

Tensors are the foundational data structure in PyTorch, akin to NumPy arrays but optimized for GPU acceleration.

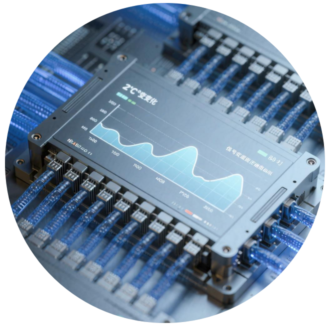
GPU Acceleration

PyTorch leverages CUDA cores to move tensor operations to the GPU, significantly speeding up computations.

GPU acceleration for tensors

Eager Execution (Define-by-Run)

PyTorch's Autograd system uses eager execution, allowing for dynamic computation graphs that simplify debugging.



Backward Pass for Gradient Calculation

The Autograd engine dynamically computes gradients during the backward pass, essential for training neural networks.

Autograd: automatic differentiation

Eager execution benefits

Backward pass for gradient calculation

03

Building the Model Lifecycle

Data handling with Dataset and DataLoader

Efficient input pipelines

Dataset and DataLoader enable efficient handling of large data for training.

Large data management

They provide mechanisms for managing large datasets with ease and efficiency.

Model construction using torch.nn.Module

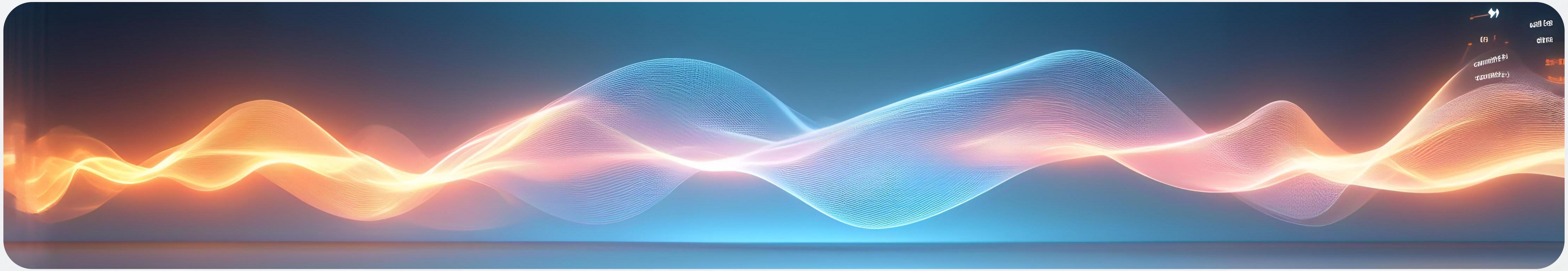
Standard modular building block

torch.nn.Module is the foundational class for building and packaging model architectures.

Flexibility in model design

It allows for the creation of complex neural network structures with modular components.

Training loop components



Loss functions and optimizers

These are essential elements for defining the training process and updating model weights.

Forward/backward flow

The training loop involves forward propagation to compute loss and backward propagation for gradient updates.

Ecosystem focus: Torchvision

CV toolkit for saascv

Torchvision provides essential datasets, models, and transforms for computer vision tasks.

Facilitates rapid iteration

It enables researchers to quickly iterate on models and experiments in the saascv domain.

04

Production and Professionalism

PyTorch Lightning for clean code

Professional wrapper for PyTorch

PyTorch Lightning simplifies code, making it cleaner and more collaborative.

Clean, collaborative code

It provides structure and automations for advanced features, reducing boilerplate.

Deployment with TorchScript and model serialization

TorchScript for production deployment

TorchScript allows PyTorch models to be serialized and run independently of Python.

Model serialization benefits

Serialization ensures models can be saved and loaded efficiently, ready for deployment.

Converting to production speed

Dynamic to static graph conversion

TorchScript converts dynamic PyTorch graphs into static graphs for optimized production speed.

Optimized for production environments

This conversion is crucial for scaling models to production, ensuring speed and efficiency.

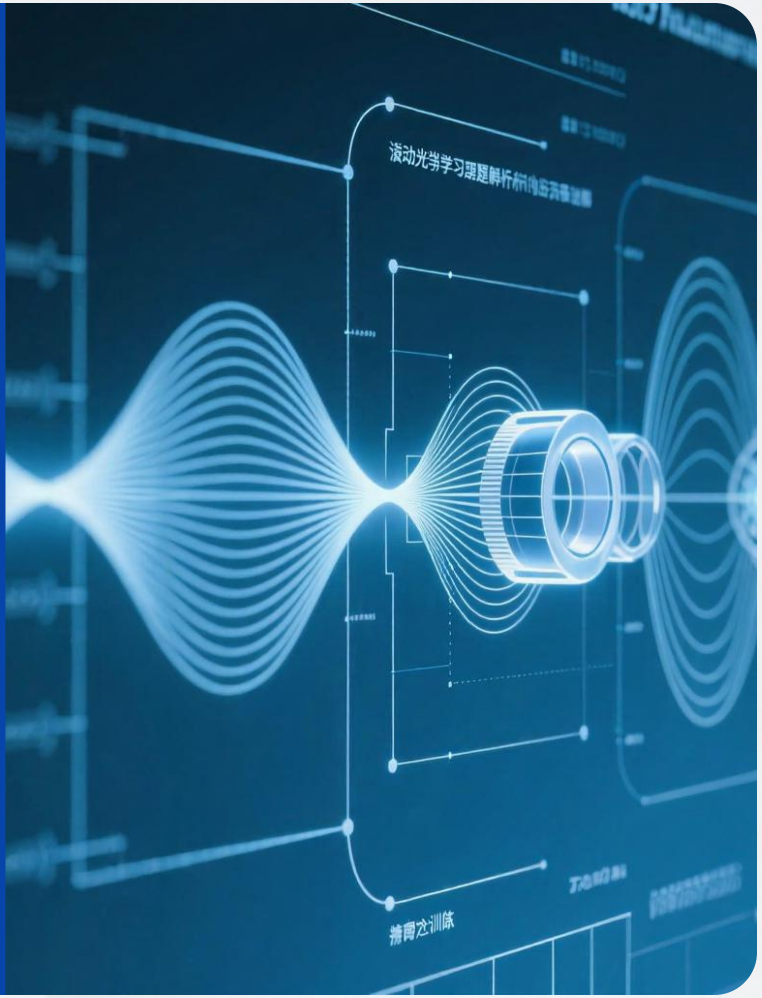
Conclusion: PyTorch's role in research

PyTorch as the research engine

PyTorch's agility and dynamic nature make it a powerful tool for rapid iteration in research.

Facilitates saascv's research

PyTorch's Python-first approach and dynamic graphs support the fast-paced research in saascv.



THE END

Thank You